**1BTS SERVICES INFORMATIQUES AUX ORGANISATIONS** **SESSION 2026**

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

| **DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE** | **N° réalisation : 2** |
|---|---|
| **Nom, prénom :** Randrianasolo Sahazaniaina | **N° candidat :** 02541491388 |
| **Épreuve ponctuelle** ☐ **Contrôle en cours de formation** ☒ | **Date :** 27 / 03 / 2026 |

**Organisation support de la réalisation professionnelle**
Section de technicien supérieur du BTS SIO option SLAM du lycée Léonard de Vinci.

**Intitulé de la réalisation professionnelle**
**SAV-App** (Application d'Aide à la Saponification à Froid)

**Période de réalisation :** 16/02/2026 au 27/03/2026..... **Lieu :** LPO Léonard de Vinci (0770934X)
**Modalité :** ☒ **Seul(e)** ☐ **En équipe**

**Compétences travaillées**
☒ Concevoir et développer une solution applicative
☒ Assurer la maintenance corrective ou évolutive d'une solution applicative
☒ Gérer les données

**Conditions de réalisation[1] (ressources fournies, résultats attendus)**
- Documentation et ressources mis à disposition sur l'ENT de l'établissement :
  - Eléments du cahier des charges : Contexte Projet SaF-Adepro.pdf
  - Fiche de calcul d'un savon : Calculateur_sapo.xlsx
  - Fiche Synthèse Saponification
  - Manuel de calcul.pdf
- API REST : https://github.com/Timomoulin/SavonApi2
- Dossiers et assets pour le maquettage
- Ensemble de documents de travail pour les étapes 5 à 9 :

**Description des ressources documentaires, matérielles et logicielles utilisées[2]**
- IDE/Editeur utilisé : **VSCode + Bruno (tests API) + IntelliJ (exéction API REST)**
- Langages utilisés : **Type Script**
- Framework utilisé : **Angular** (frontend)**+ Bootstrap** (css)
- Infrastructure : Poste de développement et test sous Ubuntu + serveur du SGBD (Debian + MariaDB)

**Modalités d'accès aux productions[3] et à leur documentation[4]**
Les documents et le code source est disponible sur le dépôt GitHub suivant :
- https://github.com/sahaza77/savonapp

[1] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.
[2] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.
[3] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.
[4] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle**
**(verso, éventuellement pages suivantes)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

**Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

## 📝 *DESCRIPTIF DE LA RÉALISATION PROFESSIONNELLE – SAVONAPP*

### 🔹 Présentation du projet

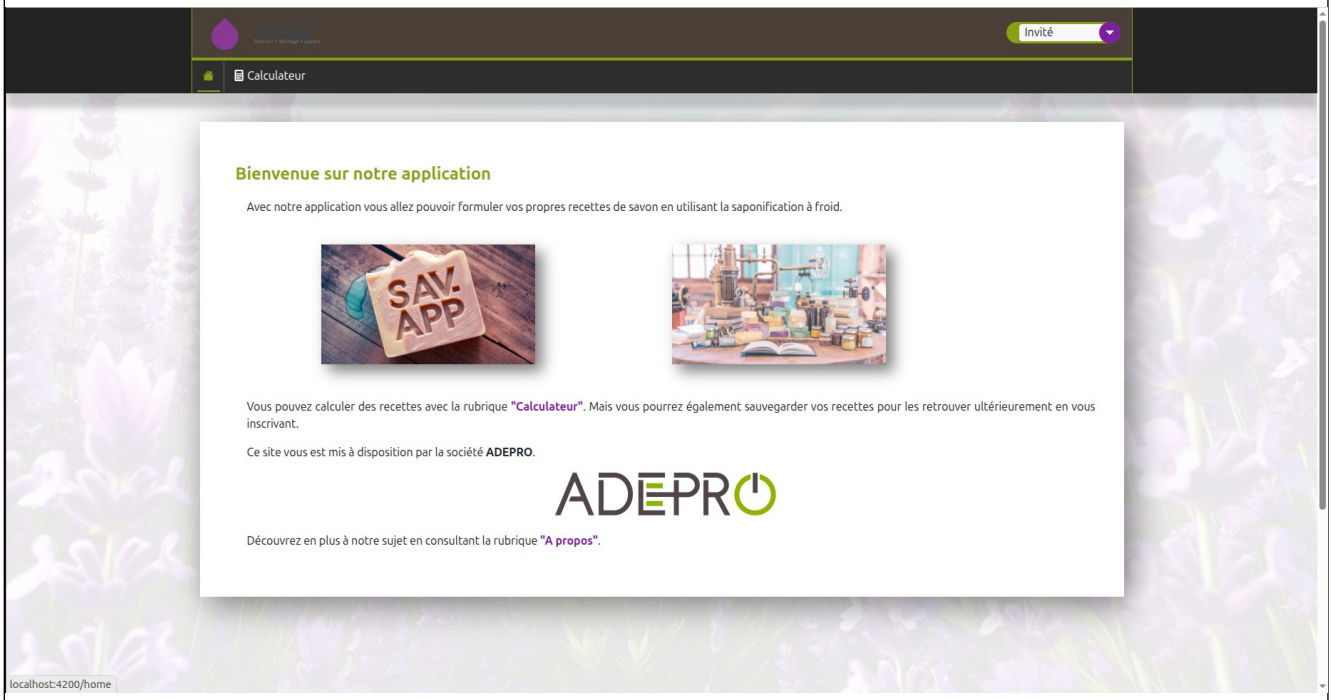
Dans le cadre de ma formation en BTS SIO option SLAM, j'ai développé une application web nommée **SavApp**.

Cette application permet de **créer, calculer et gérer des recettes de savon** en utilisant la méthode de saponification à froid.

Elle répond à un besoin métier lié à la formulation de produits cosmétiques, notamment pour la société ADEPRO.

L'utilisateur peut :

- sélectionner des ingrédients (huiles, beurres)
- définir les paramètres (surgraissage, concentration…)
- calculer automatiquement les propriétés du savon
- sauvegarder ses recettes (si connecté)
- gérer les données (admin uniquement)




👉 Page d'accueil de l'application (mode invité)

### 🔹 Environnement technique

- Frontend : Angular
- Backend : Spring Boot (API REST)

- Communication : HTTP + JSON
- Sécurité : JWT (JSON Web Token)
- Librairies : Bootstrap, Chart.js

# 🔧 DESCRIPTION DES TÂCHES RÉALISÉES

## 🔹 Tâche 1 — Structure et navigation

J'ai mis en place l'architecture Angular :

- création des composants : Header, Navbar, Footer
- configuration du routage (`app.routes.ts`)
- mise en place des pages principales

```
app.routes.ts ×
src > app > app.routes.ts > routes
import { SubscribePage } from './pages/subscribe-page/subscribe-page';
import { AccountManagerPage } from './pages/account-manager-page/account-manager-page';
import { RecipeCalculatorPage } from './pages/recipe-calculator-page/recipe-calculator-page';
import { RecipeManagerPage } from './pages/recipe-manager-page/recipe-manager-page';
import { UsersManagerPage } from './pages/users-manager-page/users-manager-page';
import { IngredientManagerPage } from './pages/ingredient-manager-page/ingredient-manager-page';
import { AboutPage } from './pages/about-page/about-page';
import { LegalNoticesPages } from './pages/legal-notices-pages/legal-notices-pages';
import { authGuard } from './guards/auth-guard';
import { AdminRecipesPage } from './pages/admin-recipes-page/admin-recipes-page';


export const routes: Routes = [
    // Redirection de la racine vers la page d'accueil :
    { path: '', pathMatch: 'full', redirectTo: 'home' },
    { path: 'home', component: HomePage },

    // Auth
    { path: 'login', component: LoginPage},
    { path: 'subscribe', component: SubscribePage },

    // Compte
    { path: 'account', component: AccountManagerPage, canActivate: [authGuard] },

    // Recettes
    { path: 'recipe-calculator', component: RecipeCalculatorPage },
    { path: 'recipe-manager', component: RecipeManagerPage, canActivate: [authGuard] },

    // Admin / gestion
    { path: 'users-manager', component: UsersManagerPage, canActivate: [authGuard] },
    { path: 'ingredients-manager', component: IngredientManagerPage, canActivate: [authGuard] },
    { path: 'admin-recipes', component: AdminRecipesPage, canActivate: [authGuard] },

    // Divers
    { path: 'about', component: AboutPage },
    { path: 'legal', component: LegalNoticesPages },

    // Fallback
    { path: '**', redirectTo: 'home' }
```

👉 Code `app.routes.ts`

## 🔹 Tâche 2 — Modélisation des données

J'ai créé les interfaces TypeScript correspondant aux données :

- Ingredient
- Recette
- LigneIngredient
- Resultat

- DTO pour l'envoi API

Cela permet de structurer les échanges avec le backend.

recette.model.ts

src > app > models > recette.model.ts > Recette > description

```typescript
import { Ingredient } from "./ingredient.model";
/**
* Détail d'un ingrédient spécifique au sein d'une recette.
*/
export interface LigneIngredient {
    quantite: number;
    pourcentage: number;
    ingredient: Ingredient;
}
/**
* Score et appréciation pour une propriété du savon (Douceur, Lavant, etc.)
*/
export interface Resultat {
    score: number;
    caracteristique: { id: number; nom: string };
    mention?: { label: string; noteMin: number; noteMax: number };
}
/**
* Modèle complet d'une recette récupérée depuis le Backend (Lecture seule).
*/
export interface Recette {
    id: number;
    titre: string;
    description: string;
    surgraissage: number;
    apportEnEau: number;
    avecSoude: boolean;
    concentrationAlcalin: number;
    qteAlcalin: number;
    ligneIngredients: LigneIngredient[];
    resultats: Resultat[];
}
```

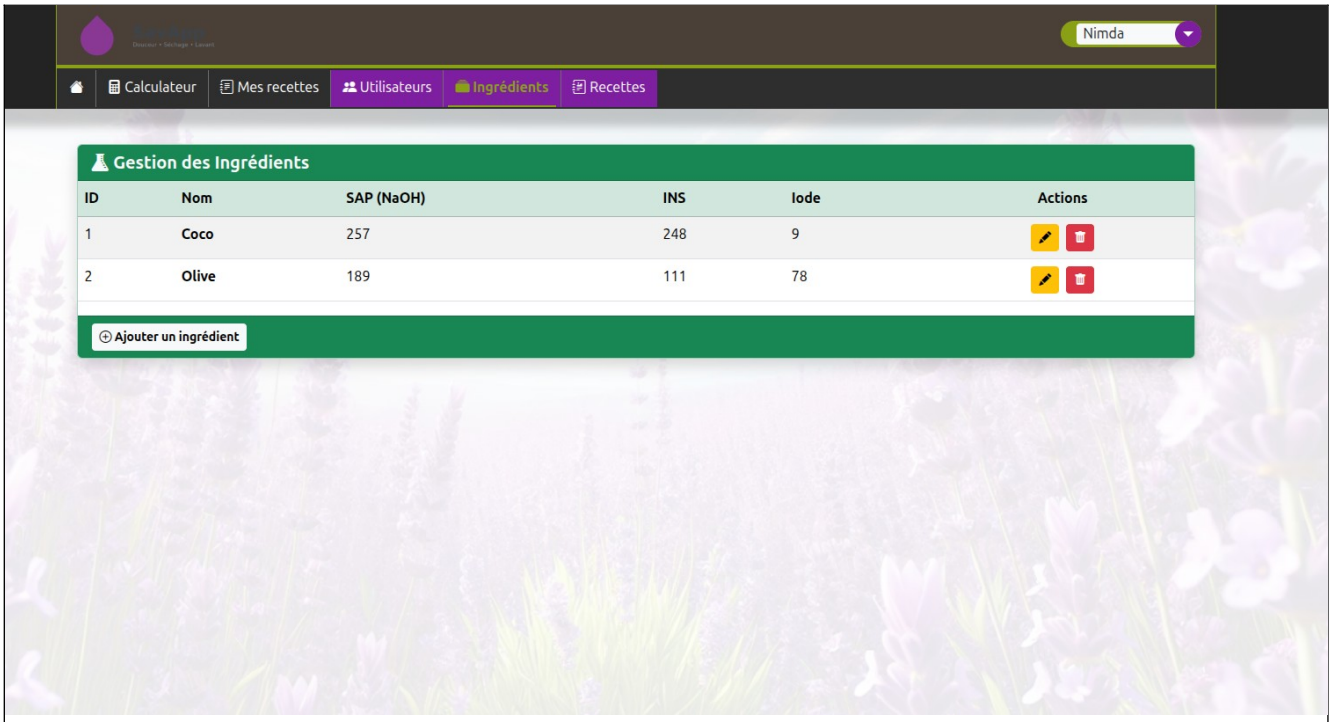
👉 Code d'un modèle `recette.model.ts`

## 🔹 Tâche 3 — Gestion des ingrédients

J'ai développé un service Angular (`IngredientService`) permettant de communiquer avec l'API.

Fonctionnalités :

- récupération des ingrédients
- affichage dans un tableau
- gestion CRUD

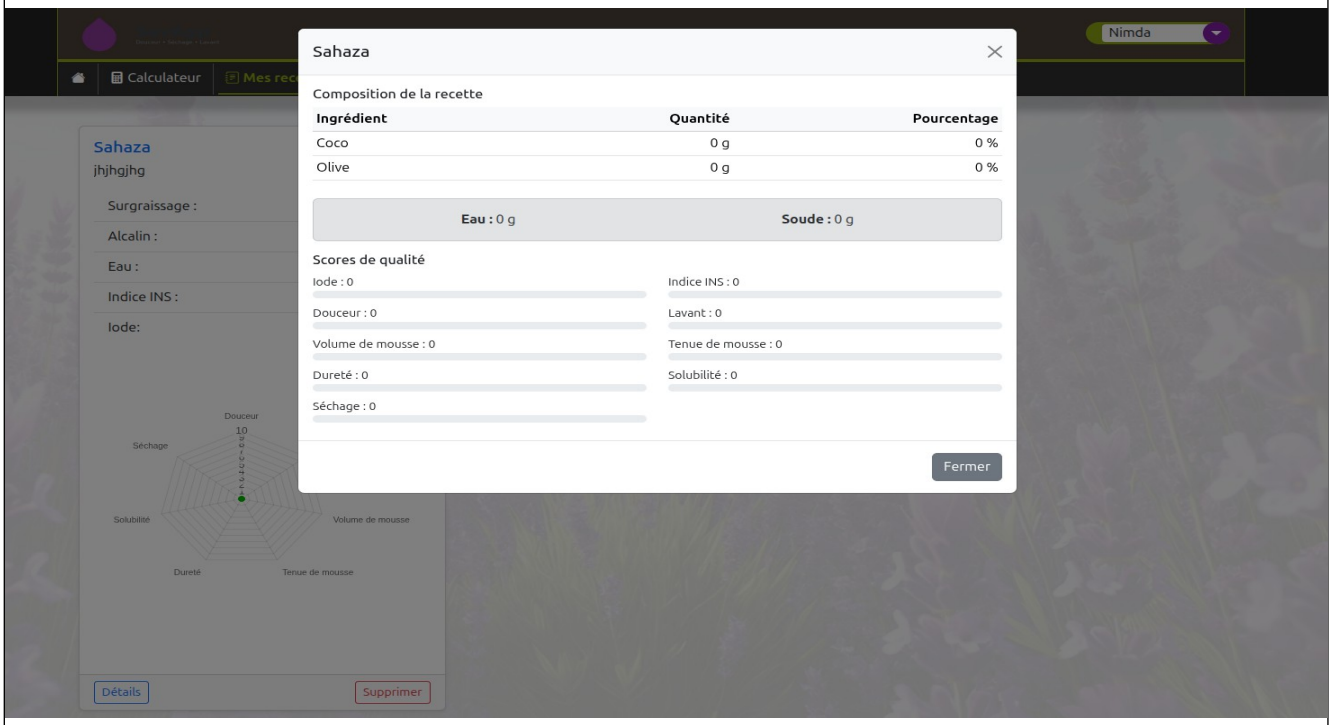



👉 Page ingrédients affichée

## 🔹 Tâche 4 — Gestion des recettes

Création du `RecetteService` :

- récupération des recettes
- suppression
- affichage sous forme de cartes



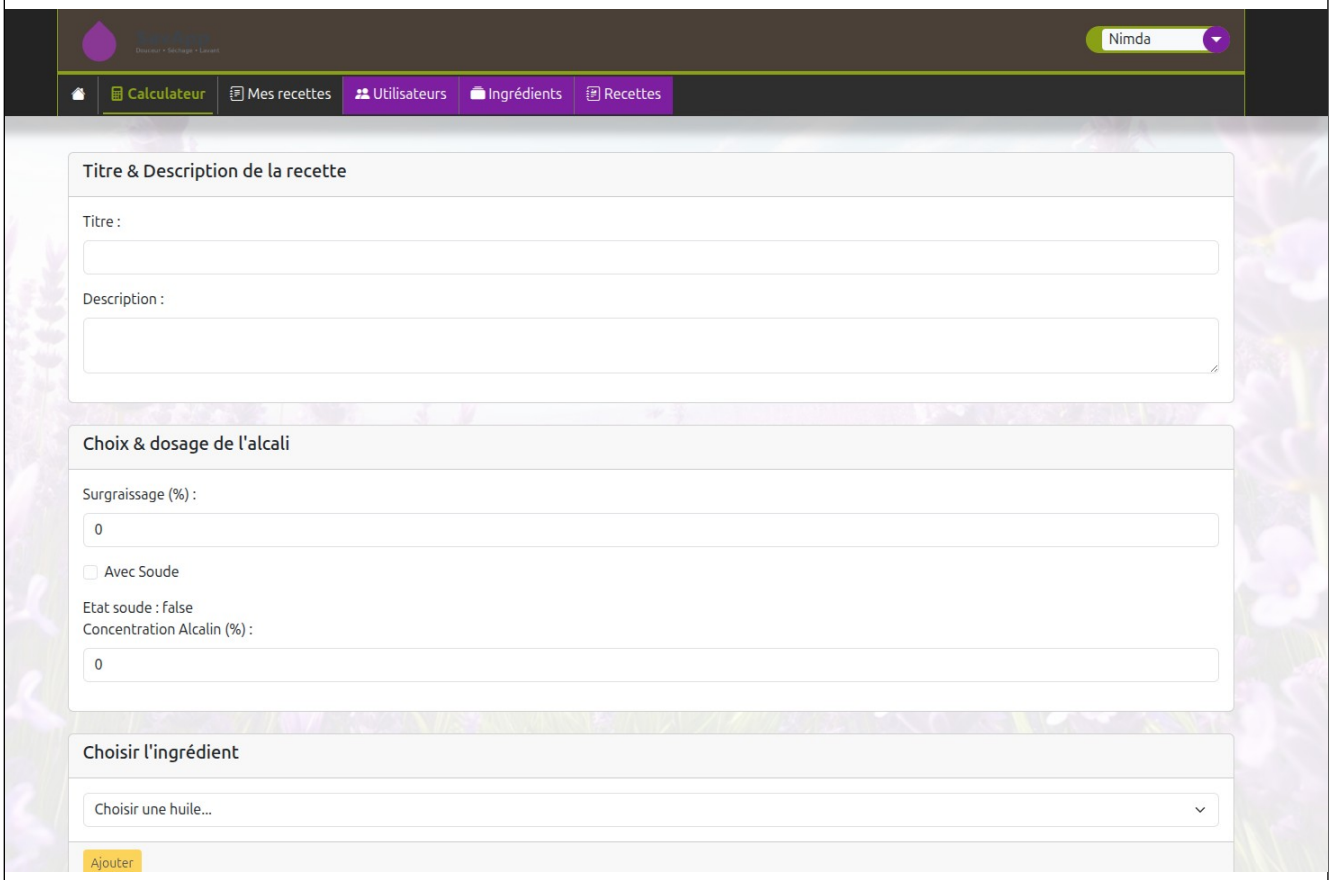
👉 Page “Mes recettes” (cards + bouton supprimer)

## 🔹 Tâche 5 — Calculateur de recette (partie principale)

J’ai développé la page **RecipeCalculatorPage**.

Fonctionnement :

1. sélection des ingrédients
2. saisie des quantités
3. calcul automatique :
    - pourcentages
    - quantité d’eau
    - quantité d’alcali
    - propriétés du savon



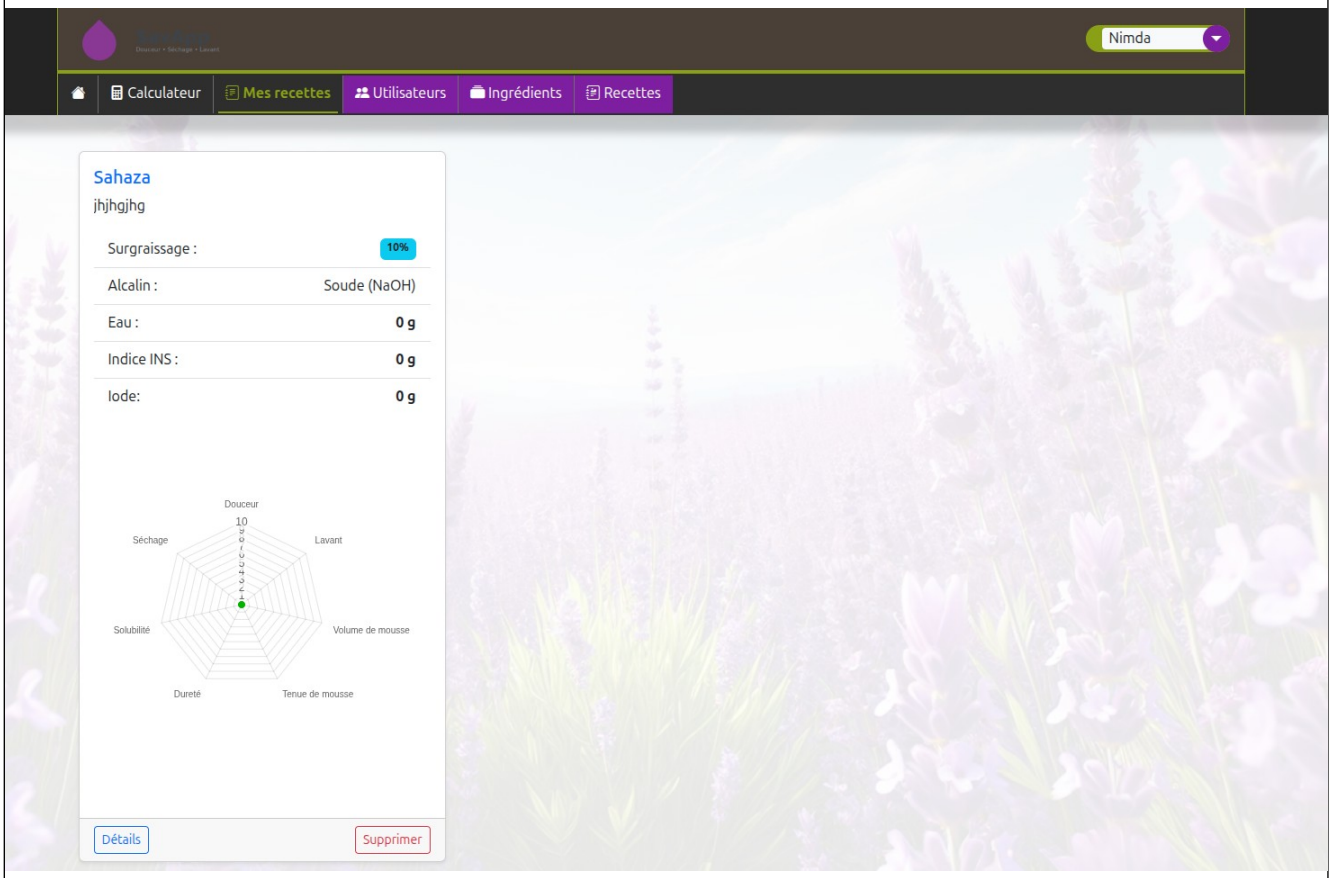

👉 Ton écran calculateur (tu me l’as envoyé ✔️)

## 🔹 Tâche 6 — Affichage des résultats

Les résultats sont affichés :

- sous forme de barres de progression
- avec des scores (douceur, dureté…)

- graphique radar (Chart.js)




👉 Résultat de calcul (barres + graphique)

## 🔹 Tâche 7 — Authentification JWT (sécurité)

J'ai mis en place un système de sécurité avec JWT :

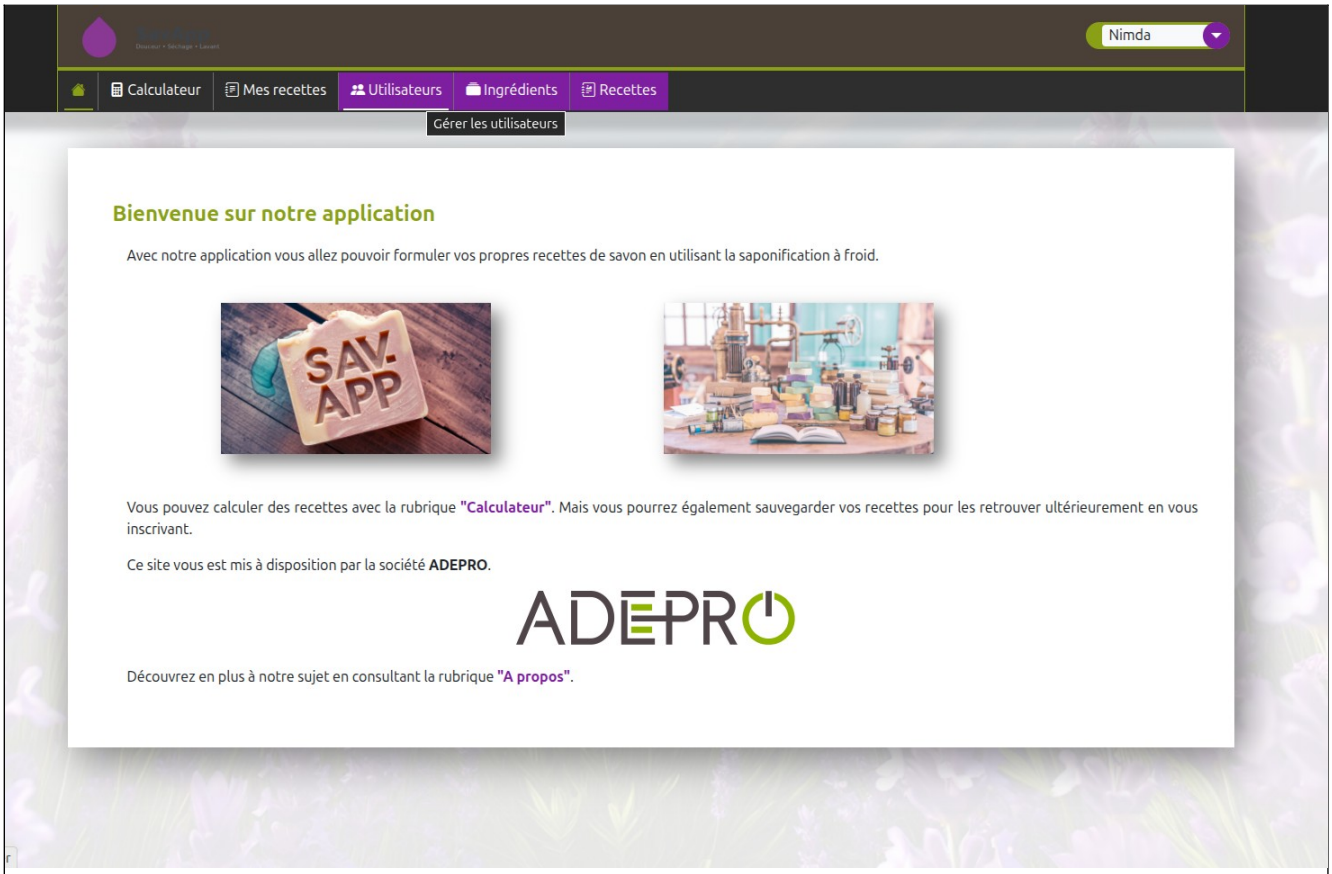
- AuthService : gestion du token
- AuthInterceptor : ajout automatique du token
- AuthGuard : protection des routes

Le token est stocké dans le navigateur et envoyé à chaque requête.

## 🔹 Tâche 8 — Gestion de session et UI dynamique

L'interface s'adapte selon l'utilisateur :

- affichage du nom dans le header
- menus différents selon le rôle
- déconnexion
- protection des pages

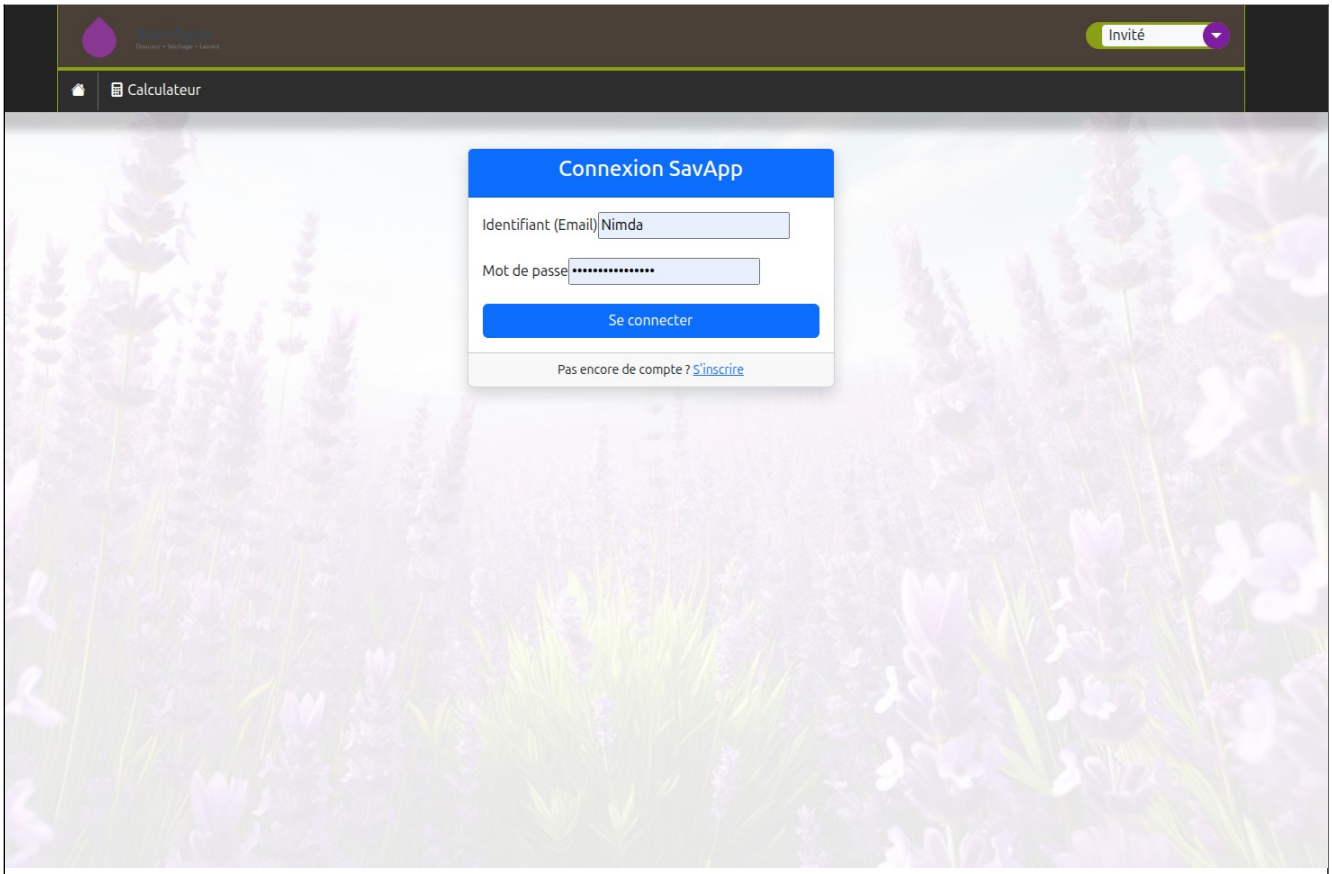



👉 Navbar en mode ADMIN (ta capture avec "Nimda" 🔥)

## 🔹 Tâche 9 — Connexion / Inscription

J'ai développé :

- page login
- page inscription
- gestion des erreurs

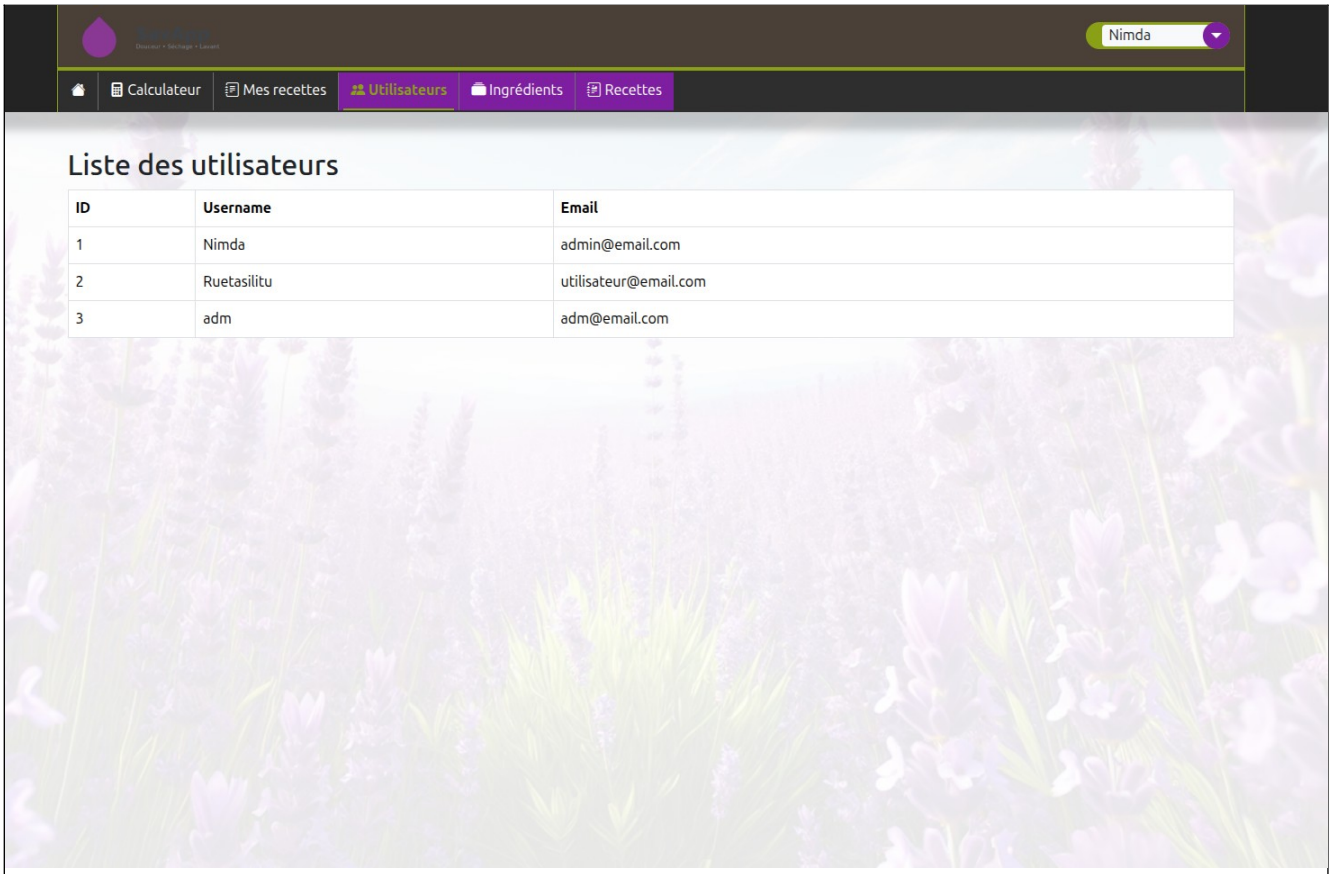



👉 Formulaire login ou inscription

## 🔹 Tâche 10 — Gestion des utilisateurs (admin)

Création d'une page admin :

- affichage des utilisateurs
- accès réservé aux admins

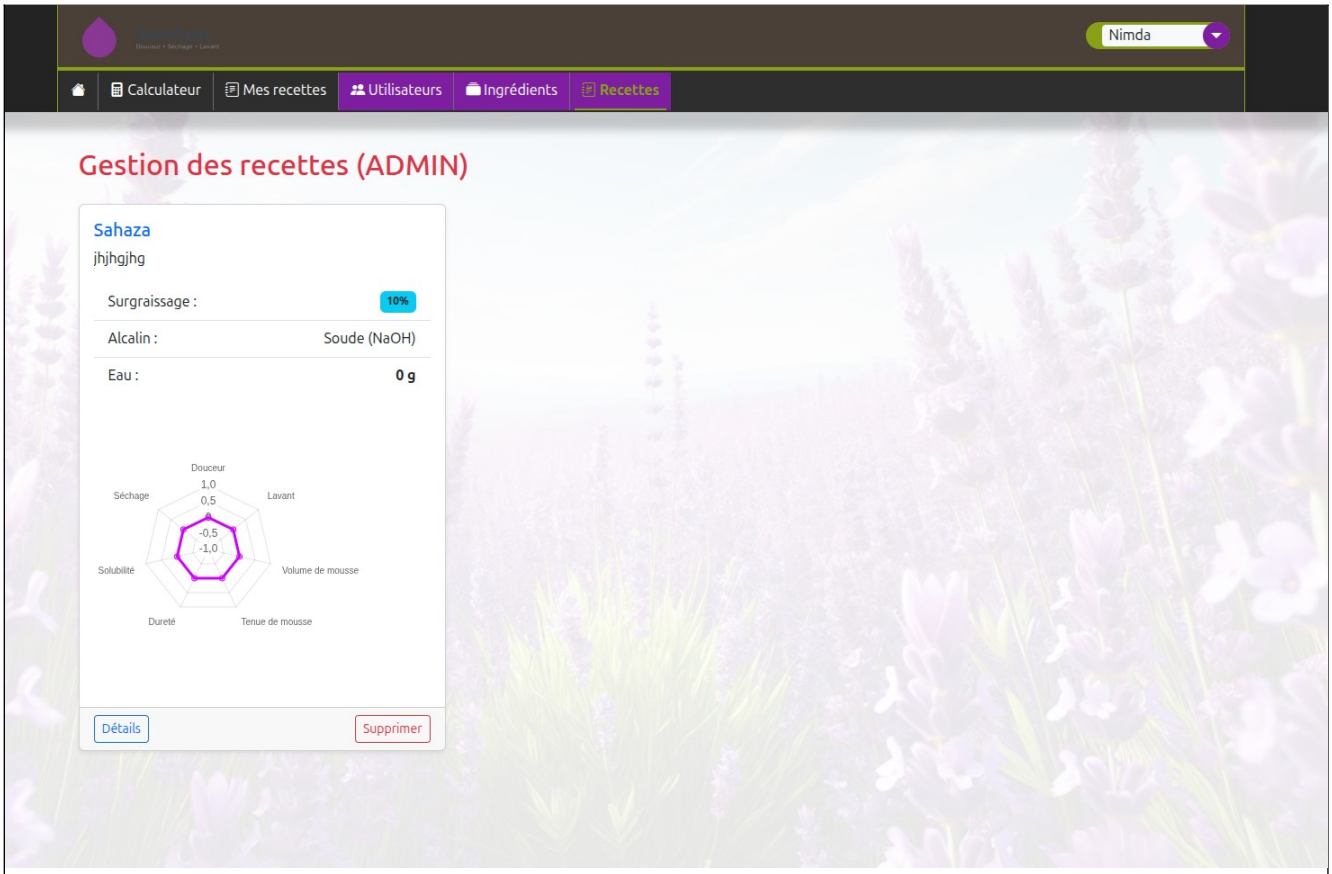



👉 Page utilisateurs

## 🔹 Tâche 11 — Gestion avancée des recettes (admin)

Création d'une page spécifique admin :

- accès à toutes les recettes
- gestion globale




👉 Page “Recettes” admin

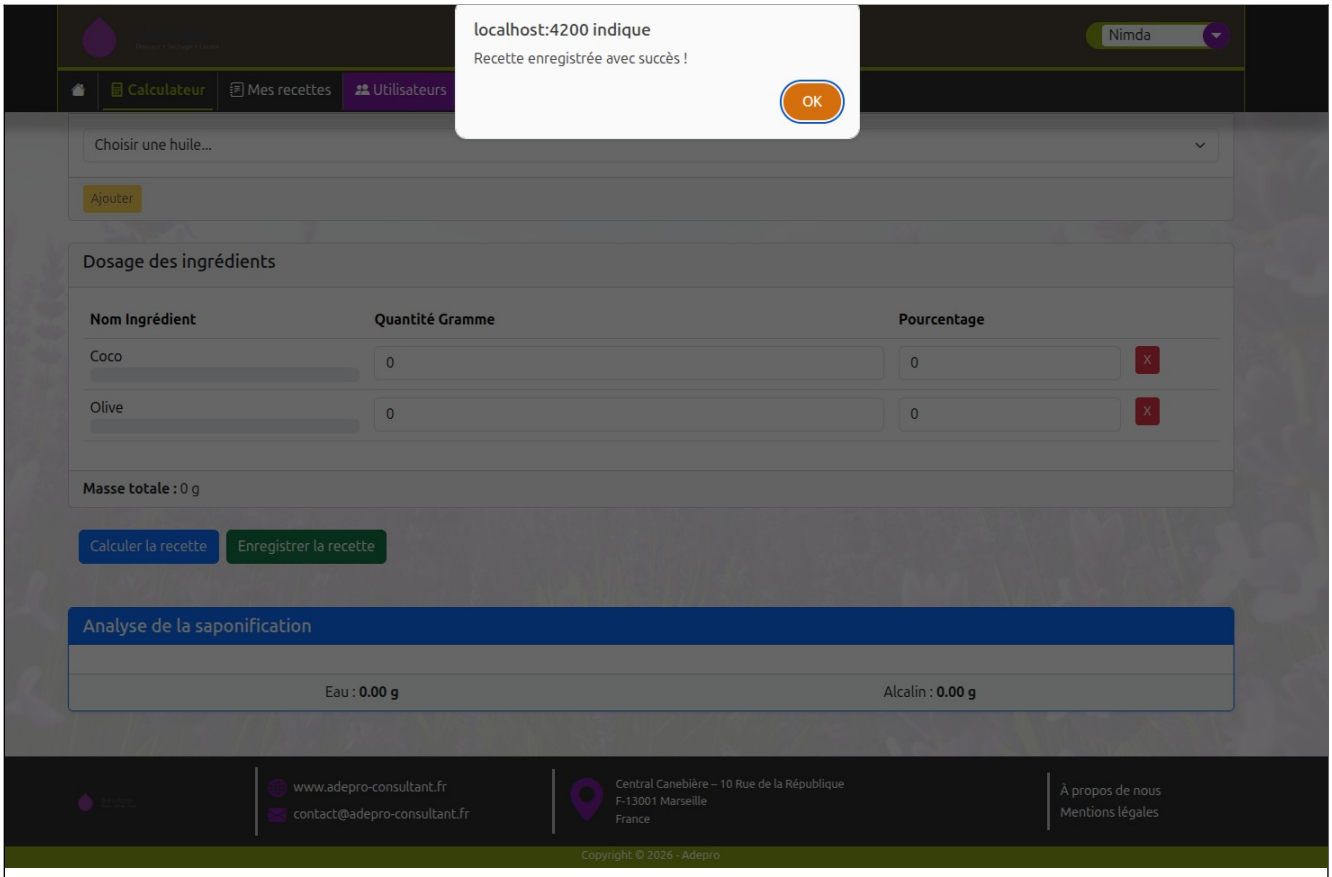
## 🔹 Tâche 12 — Adaptation du calcul selon l’utilisateur

Modification du comportement :

- invité → calcul uniquement
- connecté → possibilité d’enregistrer

Ajout :

- bouton conditionnel
- message incitant à se connecter




👉 Bouton "Enregistrer" visible connecté / caché invité